POUPON Antoine

Universidad del Rosario

Inge datos : Laboratorio 1

PARTE A

Codigo :

```
// A1
print("A1 base: " + db.getName())
printjson(db.getCollectionNames())

// A2
print("A2 restaurantes: " + db.restaurantes.countDocuments())
print("A2 clientes: " + db.clientes.countDocuments())
print("A2 productos: " + db.productos.countDocuments())
print("A2 pedidos: " + db.pedidos.countDocuments())

// A3
printjson(db.restaurantes.insertOne({
 _id: "R11",
 nombre: "Café Arriero",
 categoria: "Cafetería",
 ciudad: "Medellín",
 barrio: "Laureles",
 calificacion: 4.5,
 domicilio: true,
 ubicacion: { type: "Point", coordinates: [-75.5925, 6.2442] },
 etiquetas: ["café", "postres"]
}))

// A4
printjson(db.productos.insertMany([
 { _id: "P25", restaurante_id: "R11", nombre: "Tinto campesino", tipo: "bebida", precio: 3500,
stock: 100, disponible: true, volumen_ml: 200 },
 { _id: "P26", restaurante_id: "R11", nombre: "Torta de zanahoria", tipo: "postre", precio: 9000,
stock: 20, disponible: true }
]))

// A5
printjson(db.restaurantes.find({ ciudad: "Medellín" }).toArray())
print("A5 productos R11: " + db.productos.countDocuments({ restaurante_id: "R11" }))
```

Respuesta :

```
A1 base: saborya
[ 'clientes', 'restaurantes', 'productos', 'pedidos' ]
A2 restaurantes: 10
A2 clientes: 8
A2 productos: 24
A2 pedidos: 20
{
```

```
 acknowledged: true,
 insertedId: 'R11'
}
{
 acknowledged: true,
 insertedIds: {
  '0': 'P25',
  '1': 'P26'
 }
}
[
 {
  _id: 'R06',
  nombre: 'Verde Vivo',
  categoria: 'Vegetariana',
  ciudad: 'Medellín',
  barrio: 'El Poblado',
  calificacion: 4.4,
  domicilio: false,
  ubicacion: { type: 'Point', coordinates: [ -75.5664, 6.2088 ] },
  etiquetas: [ 'vegano', 'saludable' ],
  menu_vegano: true
 },
 {
  _id: 'R07',
  nombre: 'Bandeja Paisa Doña Rosa',
  categoria: 'Comida colombiana',
  ciudad: 'Medellín',
  barrio: 'Laureles',
  calificacion: 4.7,
  domicilio: true,
  ubicacion: { type: 'Point', coordinates: [ -75.59, 6.245 ] },
  etiquetas: [ 'bandeja paisa', 'almuerzos' ]
 },
 {
  _id: 'R08',
  nombre: 'Taco Loco',
  categoria: 'Mexicana',
  ciudad: 'Medellín',
  barrio: 'Envigado',
  calificacion: 4.1,
  domicilio: true,
  ubicacion: { type: 'Point', coordinates: [ -75.588, 6.171 ] },
  etiquetas: [ 'tacos', 'picante' ]
 },
 {
  _id: 'R11',
  nombre: 'Café Arriero',
  categoria: 'Cafetería',
  ciudad: 'Medellín',
  barrio: 'Laureles',
  calificacion: 4.5,
```

```
  domicilio: true,
  ubicacion: { type: 'Point', coordinates: [ -75.5925, 6.2442 ] },
  etiquetas: [ 'café', 'postres' ]
 }
]
```

A5 productos R11: 2

PARTE B

Codigo :

```
// B1. Comprobar tipos
print("B1 pedidos con fecha tipo Date: " + db.pedidos.countDocuments({ fecha: { $type: "date" }
}))
print("B1 pedidos con fecha que NO es Date: " + db.pedidos.countDocuments({ fecha: { $not: {
$type: "date" } } }))
print("B1 productos con precio guardado como texto: " + db.productos.countDocuments({
precio: { $type: "string" } }))

// B3. Colección resenas con $jsonSchema
db.createCollection("resenas", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["restaurante_id", "cliente", "calificacion", "texto", "fecha"],
   properties: {
    restaurante_id: { bsonType: "string" },
    cliente: {
     bsonType: "object",
     required: ["id", "nombre"],
     properties: {
      id: { bsonType: "string" },
      nombre: { bsonType: "string" }
     }
    },
    calificacion: { bsonType: "int", minimum: 1, maximum: 5 },
    texto: { bsonType: "string", minLength: 5 },
    fecha: { bsonType: "date" },
    fotos: {
     bsonType: "array",
     maxItems: 4,
     items: { bsonType: "string" }
    }
   }
  }
 }
})

// B4. Cuatro reseñas válidas en una sola operación
db.resenas.insertMany([
```

{ restaurante_id: "R02", cliente: { id: "C01", nombre: "Laura Gómez" }, calificacion: 5, texto: "El ramen llegó caliente y bien empacado", fecha: new Date("2026-10-15T20:10:00-05:00") },
{ restaurante_id: "R02", cliente: { id: "C08", nombre: "Julián Castro" }, calificacion: 4, texto: "Buen sushi, pero tardó 50 minutos", fecha: new Date("2026-10-16T21:30:00-05:00") },
{ restaurante_id: "R10", cliente: { id: "C07", nombre: "Daniela Martínez" }, calificacion: 5, texto: "La cazuela es abundante", fotos: ["https://cdn.saborya.co/r/301.jpg", "https://cdn.saborya.co/r/302.jpg"], fecha: new Date("2026-10-17T13:45:00-05:00") },
{ restaurante_id: "R04", cliente: { id: "C03", nombre: "Valentina Ríos" }, calificacion: 2, texto: "Las papas llegaron frías", fecha: new Date("2026-10-18T19:05:00-05:00") }
])

Respuesta :

```
B1 pedidos con fecha tipo Date: 20
B1 pedidos con fecha que NO es Date: 0
B1 productos con precio guardado como texto: 0
{
 acknowledged: true,
 insertedIds: {
  '0': ObjectId('6aeb5668a85a1ddf5a279958'),
  '1': ObjectId('6aeb5668a85a1ddf5a279959'),
  '2': ObjectId('6aeb5668a85a1ddf5a27995a'),
  '3': ObjectId('6aeb5668a85a1ddf5a27995b')
 }
}
```

// B2. Guardaría las reseñas en una colección propia (resenas), no dentro del documento
// del restaurante. Patrón de acceso: la pantalla muestra el promedio, el total y las más
// recientes; no necesita cargar todas las reseñas cada vez que se abre un restaurante.
// Con una colección propia puedo leer solo las últimas (ordenadas por fecha, con un
// índice por restaurante_id y fecha) y paginarlas. Límite de 16 MB: un documento no puede
// superar 16 MB; las reseñas de un restaurante popular crecen sin límite, así que
// incrustarlas haría crecer el documento sin control y lo acercaría a ese tope, además de
// hacerlo pesado en cada lectura. En cada reseña copiaría solo cliente.id y cliente.nombre:
// el nombre se muestra junto a la reseña, y copiarlo evita una consulta extra al leer; el
// id permite buscar las reseñas de un cliente. No copiaría email, puntos ni direcciones:
// no se muestran y cambian con frecuencia. El promedio y el total se guardan como resumen
// en el restaurante (B7) para no recalcularlos en cada pantalla.

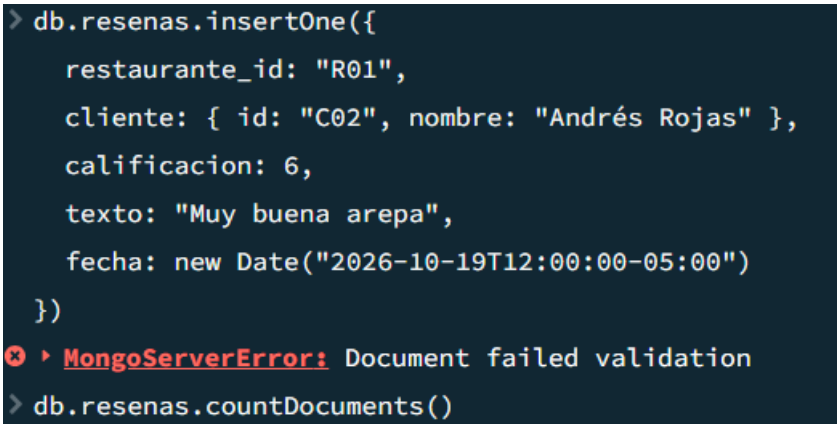
Codigo :

```
//B5
db.resenas.insertOne({
 restaurante_id: "R01",
 cliente: { id: "C02", nombre: "Andrés Rojas" },
 calificacion: 6,
 texto: "Muy buena arepa",
 fecha: new Date("2026-10-19T12:00:00-05:00")
})
```

Respuesta :

```
db.resenas.insertOne({
 restaurante_id: "R01",
 cliente: { id: "C02", nombre: "Andrés Rojas" },
 calificacion: 6,
 texto: "Muy buena arepa",
 fecha: new Date("2026-10-19T12:00:00-05:00")
})
MongoServerError: Document failed validation

// B5. Resultado: MongoServerError: Document failed validation (código 121).
// Explicación: la regla $jsonSchema de la colección resenas exige que calificacion sea un
// entero entre 1 y 5. El valor 6 supera la propiedad "maximum: 5" del campo calificacion,
// así que MongoDB rechaza el documento completo y no se inserta nada. Los demás campos
// (restaurante_id, cliente, texto, fecha) sí cumplían la regla, el único que falló fue calificacion.
```



Codigo :

```
// B6. Reseñas de Laura Gómez y reseñas con al menos una foto
printjson(db.resenas.find({ "cliente.id": "C01" }).toArray())
printjson(db.resenas.find({ "fotos.0": { $exists: true } }).toArray())

// B7. Denormalización controlada en R02 (promedio de 5 y 4 = 4.5, total = 2)
printjson(db.restaurantes.updateOne(
 { _id: "R02" },
 { $set: { resumen_resenas: { promedio: 4.5, total: 2 } } }
))

// B8. Restaurantes distintos con reseñas
printjson(db.resenas.distinct("restaurante_id"))
```

Respuesta :





PARTE C

Codigo :

```
// ===== PARTE C =====
use("saborya")
print("Base actual: " + db.getName())

// C1. Platos disponibles con stock < 20 (nombre y stock, sin _id, de menor a mayor stock)
// Desempate por nombre para que el orden sea siempre el mismo.
printjson(db.productos.find(
 { tipo: "plato", disponible: true, stock: { $lt: 20 } },
 { _id: 0, nombre: 1, stock: 1 }
).sort({ stock: 1, nombre: 1 }).toArray())

// C2. Pedidos con total entre 50.000 y 80.000 pagados en efectivo o nequi
printjson(db.pedidos.find(
 { total: { $gte: 50000, $lte: 80000 }, metodo_pago: { $in: ["efectivo", "nequi"] } },
 { _id: 0, total: 1, metodo_pago: 1 }
).toArray())

// C3. Los 3 restaurantes de Bogotá con mejor calificación
printjson(db.restaurantes.find(
 { ciudad: "Bogotá" },
 { _id: 0, nombre: 1, calificacion: 1 }
).sort({ calificacion: -1 }).limit(3).toArray())

// C4. Clientes de Bogotá con más de 100 puntos
printjson(db.clientes.find({ ciudad: "Bogotá", puntos: { $gt: 100 } }).toArray())

// C5. Pedidos del 12 al 14 de oct 2026 (hora Colombia, ambos días incluidos) no entregados
printjson(db.pedidos.find({
 fecha: {
```

```
    $gte: new Date("2026-10-12T00:00:00-05:00"),
    $lt: new Date("2026-10-15T00:00:00-05:00")
  },
  estado: { $ne: "entregado" }
}).toArray())

// C6. Ítem con precio < 10000 Y cantidad >= 2 en ese mismo ítem
print("C6 con $elemMatch:")
printjson(db.pedidos.find({
  items: { $elemMatch: { precio: { $lt: 10000 }, cantidad: { $gte: 2 } } }
}).toArray())

print("C6 sin $elemMatch:")
printjson(db.pedidos.find({
  "items.precio": { $lt: 10000 },
  "items.cantidad": { $gte: 2 }
}).toArray())

// C7. Catálogo del más caro al más barato, 6 por página, página 2 (estable con precios
repetidos)
printjson(db.productos.find(
  {},
  { nombre: 1, precio: 1 }
).sort({ precio: -1, _id: 1 }).skip(6).limit(6).toArray())

// C8. Restaurantes que tienen el campo horario o el campo menu_vegano
printjson(db.restaurantes.find({
  $or: [{ horario: { $exists: true } }, { menu_vegano: { $exists: true } }]
}).toArray())
```

Respuesta :

C1 hasta C3

```
[
  { nombre: 'Cazuela de mariscos', stock: 10 },
  { nombre: 'Picada para dos', stock: 12 },
  { nombre: 'Arroz con coco y pescado', stock: 14 },
  { nombre: 'Combo Sake', stock: 15 },
  { nombre: 'Costillas BBQ', stock: 15 },
  { nombre: 'Hamburguesa de lentejas', stock: 18 }
]
[
  { metodo_pago: 'nequi', total: 60000 },
  { metodo_pago: 'efectivo', total: 76000 },
  { metodo_pago: 'nequi', total: 70000 },
  { metodo_pago: 'efectivo', total: 64000 },
  { metodo_pago: 'nequi', total: 67000 },
  { metodo_pago: 'efectivo', total: 78000 }
]
[
  { nombre: 'Sushi Nikko', calificacion: 4.8 },
  { nombre: 'Arepas La 19', calificacion: 4.6 },
  { nombre: 'Ajiaco & Co', calificacion: 4.5 }
]
```

POUPON Antoine

C4 hasta C5

```
[
 {
  _id: 'C01',
  nombre: 'Laura Gómez',
  email: 'laura.gomez@example.com',
  ciudad: 'Bogotá',
  fecha_registro: 2026-08-14T00:00:00.000Z,
  puntos: 120,
  direcciones: [
   { alias: 'Casa', direccion: 'Calle 72 # 10-34', barrio: 'Chapinero' }
  ],
  preferencias: [ 'japonesa', 'colombiana' ]
 },
 {
  _id: 'C02',
  nombre: 'Andrés Rojas',
  email: 'andres.rojas@example.com',
  ciudad: 'Bogotá',
  fecha_registro: 2025-11-02T00:00:00.000Z,
  puntos: 340,
  direcciones: [
   { alias: 'Casa', direccion: 'Carrera 15 # 93-20', barrio: 'Chicó' },
   { alias: 'Oficina', direccion: 'Calle 26 # 69-76', barrio: 'Salitre' }
  ],
  preferencias: [ 'colombiana', 'italiana' ]
 },
 {
  _id: 'C08',
  nombre: 'Julián Castro',
  email: 'julian.castro@example.com',
  ciudad: 'Bogotá',
  fecha_registro: 2024-12-01T00:00:00.000Z,
  puntos: 600,
  direcciones: [ { alias: 'Casa', direccion: 'Calle 85 # 12-30', barrio: 'Chicó' } ],
  preferencias: [ 'japonesa', 'italiana', 'hamburguesas' ]
 }
]
```

```
[
 {
  _id: 'PD015',
  cliente: { id: 'C08', nombre: 'Julián Castro' },
  restaurante: { id: 'R04', nombre: 'Burger Norte', ciudad: 'Bogotá' },
  fecha: 2026-10-12T23:30:00.000Z,
  estado: 'en_camino',
  metodo_pago: 'tarjeta',
  items: [
   {
```

```
      producto_id: 'P10',
      nombre: 'Hamburguesa clásica',
      precio: 22000,
      cantidad: 1
    },
    {
      producto_id: 'P11',
      nombre: 'Papas a la francesa',
      precio: 8000,
      cantidad: 1
    }
  ],
  total: 30000
},
{
  _id: 'PD016',
  cliente: { id: 'C05', nombre: 'Mariana López' },
  restaurante: { id: 'R07', nombre: 'Bandeja Paisa Doña Rosa', ciudad: 'Medellín' },
  fecha: 2026-10-12T18:00:00.000Z,
  estado: 'pendiente',
  metodo_pago: 'efectivo',
  items: [
    {
      producto_id: 'P16',
      nombre: 'Bandeja paisa',
      precio: 34000,
      cantidad: 1
    }
  ],
  total: 34000
},
{
  _id: 'PD017',
  cliente: { id: 'C01', nombre: 'Laura Gómez' },
  restaurante: { id: 'R02', nombre: 'Sushi Nikko', ciudad: 'Bogotá' },
  fecha: 2026-10-14T01:20:00.000Z,
  estado: 'en_camino',
  metodo_pago: 'tarjeta',
  items: [
    {
      producto_id: 'P04',
      nombre: 'Combo Sake',
      precio: 41000,
      cantidad: 2
    }
  ],
  total: 82000
},
{
  _id: 'PD018',
  cliente: { id: 'C07', nombre: 'Daniela Martínez' },
  restaurante: { id: 'R09', nombre: 'Brasa Caribe', ciudad: 'Barranquilla' },
```

```
  fecha: 2026-10-14T02:00:00.000Z,
  estado: 'pendiente',
  metodo_pago: 'tarjeta',
  items: [
   {
    producto_id: 'P21',
    nombre: 'Costillas BBQ',
    precio: 45000,
    cantidad: 1
   },
   {
    producto_id: 'P20',
    nombre: 'Picada para dos',
    precio: 58000,
    cantidad: 1
   }
  ],
  total: 103000
 },
 {
  _id: 'PD020',
  cliente: { id: 'C04', nombre: 'Camilo Torres' },
  restaurante: { id: 'R06', nombre: 'Verde Vivo', ciudad: 'Medellín' },
  fecha: 2026-10-15T00:10:00.000Z,
  estado: 'cancelado',
  metodo_pago: 'nequi',
  items: [
   {
    producto_id: 'P15',
    nombre: 'Hamburguesa de lentejas',
    precio: 24000,
    cantidad: 2
   }
  ],
  total: 48000
 }
]



[
 {
  _id: 'PD001',
  cliente: { id: 'C01', nombre: 'Laura Gómez' },
  restaurante: { id: 'R02', nombre: 'Sushi Nikko', ciudad: 'Bogotá' },
  fecha: 2026-10-02T00:30:00.000Z,
  estado: 'entregado',
  metodo_pago: 'tarjeta',
  items: [
   {
    producto_id: 'P04',
    nombre: 'Combo Sake',
```

```
      precio: 41000,
      cantidad: 1
    },
    { producto_id: 'P06', nombre: 'Té verde', precio: 7000, cantidad: 2 }
  ],
  total: 55000
 },
 {
  _id: 'PD002',
  cliente: { id: 'C02', nombre: 'Andrés Rojas' },
  restaurante: { id: 'R01', nombre: 'Arepas La 19', ciudad: 'Bogotá' },
  fecha: 2026-10-01T13:15:00.000Z,
  estado: 'entregado',
  metodo_pago: 'efectivo',
  items: [
   {
    producto_id: 'P01',
    nombre: 'Arepa de choclo con queso',
    precio: 9500,
    cantidad: 2
   },
   {
    producto_id: 'P03',
    nombre: 'Jugo de lulo',
    precio: 6000,
    cantidad: 1
   }
  ],
  total: 25000
 },
 {
  _id: 'PD004',
  cliente: { id: 'C03', nombre: 'Valentina Ríos' },
  restaurante: { id: 'R04', nombre: 'Burger Norte', ciudad: 'Bogotá' },
  fecha: 2026-10-03T18:05:00.000Z,
  estado: 'cancelado',
  metodo_pago: 'nequi',
  items: [
   {
    producto_id: 'P10',
    nombre: 'Hamburguesa clásica',
    precio: 22000,
    cantidad: 2
   },
   {
    producto_id: 'P11',
    nombre: 'Papas a la francesa',
    precio: 8000,
    cantidad: 2
   }
  ],
  total: 60000
```

```
 },
 {
  _id: 'PD008',
  cliente: { id: 'C07', nombre: 'Daniela Martínez' },
  restaurante: { id: 'R10', nombre: 'Mar y Coco', ciudad: 'Barranquilla' },
  fecha: 2026-10-05T18:30:00.000Z,
  estado: 'entregado',
  metodo_pago: 'nequi',
  items: [
   {
    producto_id: 'P22',
    nombre: 'Cazuela de mariscos',
    precio: 52000,
    cantidad: 1
   },
   {
    producto_id: 'P24',
    nombre: 'Limonada de coco',
    precio: 9000,
    cantidad: 2
   }
  ],
  total: 70000
 },
 {
  _id: 'PD014',
  cliente: { id: 'C03', nombre: 'Valentina Ríos' },
  restaurante: { id: 'R01', nombre: 'Arepas La 19', ciudad: 'Bogotá' },
  fecha: 2026-10-11T14:00:00.000Z,
  estado: 'entregado',
  metodo_pago: 'nequi',
  items: [
   {
    producto_id: 'P02',
    nombre: 'Arepa rellena de carne',
    precio: 14000,
    cantidad: 1
   },
   {
    producto_id: 'P03',
    nombre: 'Jugo de lulo',
    precio: 6000,
    cantidad: 2
   }
  ],
  total: 26000
 }
]


{
  _id: 'PD001',
```

```
  cliente: { id: 'C01', nombre: 'Laura Gómez' },
  restaurante: { id: 'R02', nombre: 'Sushi Nikko', ciudad: 'Bogotá' },
  fecha: 2026-10-02T00:30:00.000Z,
  estado: 'entregado',
  metodo_pago: 'tarjeta',
  items: [
   {
    producto_id: 'P04',
    nombre: 'Combo Sake',
    precio: 41000,
    cantidad: 1
   },
   { producto_id: 'P06', nombre: 'Té verde', precio: 7000, cantidad: 2 }
  ],
  total: 55000
 },
 {
  _id: 'PD002',
  cliente: { id: 'C02', nombre: 'Andrés Rojas' },
  restaurante: { id: 'R01', nombre: 'Arepas La 19', ciudad: 'Bogotá' },
  fecha: 2026-10-01T13:15:00.000Z,
  estado: 'entregado',
  metodo_pago: 'efectivo',
  items: [
   {
    producto_id: 'P01',
    nombre: 'Arepa de choclo con queso',
    precio: 9500,
    cantidad: 2
   },
   {
    producto_id: 'P03',
    nombre: 'Jugo de lulo',
    precio: 6000,
    cantidad: 1
   }
  ],
  total: 25000
 },
 {
  _id: 'PD004',
  cliente: { id: 'C03', nombre: 'Valentina Ríos' },
  restaurante: { id: 'R04', nombre: 'Burger Norte', ciudad: 'Bogotá' },
  fecha: 2026-10-03T18:05:00.000Z,
  estado: 'cancelado',
  metodo_pago: 'nequi',
  items: [
   {
    producto_id: 'P10',
    nombre: 'Hamburguesa clásica',
    precio: 22000,
    cantidad: 2
```

```
    },
    {
      producto_id: 'P11',
      nombre: 'Papas a la francesa',
      precio: 8000,
      cantidad: 2
    }
  ],
  total: 60000
 },
```

// Explicación: sin $elemMatch, cada condición se evalúa por separado y puede cumplirla un ítem distinto. // PD005 aparece de más: la Arepa con hogao cuesta 8000 (precio < 10000, pero cantidad 1) y la // Bandeja paisa tiene cantidad 2 (pero cuesta 34000). Ningún ítem cumple las dos condiciones a la vez. // $elemMatch obliga a que las dos condiciones las cumpla el mismo elemento del arreglo.

C7 hasta C8

```
{
  _id: 'PD005',
  cliente: { id: 'C04', nombre: 'Camilo Torres' },
  restaurante: { id: 'R07', nombre: 'Bandeja Paisa Doña Rosa', ciudad: 'Medellín' },
  fecha: 2026-10-03T17:40:00.000Z,
  estado: 'entregado',
  metodo_pago: 'efectivo',
  items: [
   {
     producto_id: 'P16',
     nombre: 'Bandeja paisa',
     precio: 34000,
     cantidad: 2
   },
   {
     producto_id: 'P17',
     nombre: 'Arepa con hogao',
     precio: 8000,
     cantidad: 1
   }
  ],
  total: 76000
 },
 {
  _id: 'PD008',
  cliente: { id: 'C07', nombre: 'Daniela Martínez' },
  restaurante: { id: 'R10', nombre: 'Mar y Coco', ciudad: 'Barranquilla' },
  fecha: 2026-10-05T18:30:00.000Z,
  estado: 'entregado',
  metodo_pago: 'nequi',
  items: [
```

```
    {
      producto_id: 'P22',
      nombre: 'Cazuela de mariscos',
      precio: 52000,
      cantidad: 1
    },
    {
      producto_id: 'P24',
      nombre: 'Limonada de coco',
      precio: 9000,
      cantidad: 2
    }
  ],
  total: 70000
 },
 {
  _id: 'PD014',
  cliente: { id: 'C03', nombre: 'Valentina Ríos' },
  restaurante: { id: 'R01', nombre: 'Arepas La 19', ciudad: 'Bogotá' },
  fecha: 2026-10-11T14:00:00.000Z,
  estado: 'entregado',
  metodo_pago: 'nequi',
  items: [
    {
      producto_id: 'P02',
      nombre: 'Arepa rellena de carne',
      precio: 14000,
      cantidad: 1
    },
    {
      producto_id: 'P03',
      nombre: 'Jugo de lulo',
      precio: 6000,
      cantidad: 2
    }
  ],
  total: 26000
 }
]

[
 { _id: 'P05', nombre: 'Ramen tonkotsu', precio: 36000 },
 { _id: 'P16', nombre: 'Bandeja paisa', precio: 34000 },
 { _id: 'P07', nombre: 'Pizza hawaiana mediana', precio: 32000 },
 { _id: 'P09', nombre: 'Lasaña boloñesa', precio: 29000 },
 { _id: 'P12', nombre: 'Ajiaco santafereño', precio: 28000 },
 { _id: 'P14', nombre: 'Bowl de quinua', precio: 26000 }
]
```

```
[
 {
  _id: 'R02',
  nombre: 'Sushi Nikko',
  categoria: 'Japonesa',
  ciudad: 'Bogotá',
  barrio: 'Usaquén',
  calificacion: 4.8,
  domicilio: true,
  ubicacion: { type: 'Point', coordinates: [ -74.0305, 4.695 ] },
  etiquetas: [ 'sushi', 'ramen' ],
  horario: { apertura: '12:00', cierre: '22:00' },
  resumen_resenas: { promedio: 4.5, total: 2 }
 },
 {
  _id: 'R06',
  nombre: 'Verde Vivo',
  categoria: 'Vegetariana',
  ciudad: 'Medellín',
  barrio: 'El Poblado',
  calificacion: 4.4,
  domicilio: false,
  ubicacion: { type: 'Point', coordinates: [ -75.5664, 6.2088 ] },
  etiquetas: [ 'vegano', 'saludable' ],
  menu_vegano: true
 }
]
```

PARTE D

Codigo :

```
// ===== PARTE D · Bloque 1 (D1 a D3) =====
use("saborya")
print("Base actual: " + db.getName())

// D1. Subir 1.000 pesos a los platos del R07 (ejecutar UNA sola vez)
printjson(db.productos.updateMany(
 { restaurante_id: "R07", tipo: "plato" },
 { $inc: { precio: 1000 } }
))
printjson(db.productos.find({ restaurante_id: "R07" }, { nombre: 1, tipo: 1, precio: 1 }).toArray())

// D2. PD018 a en_camino solo si está pendiente (ejecutar dos veces)
print("D2 ejecución 1:")
printjson(db.pedidos.updateOne(
 { _id: "PD018", estado: "pendiente" },
 { $set: { estado: "en_camino" } }
))
print("D2 ejecución 2:")
printjson(db.pedidos.updateOne(
```

```
 { _id: "PD018", estado: "pendiente" },
 { $set: { estado: "en_camino" } }
))

// D3. Agregar "japonesa" a las preferencias de C03 sin repetir (ejecutar dos veces)
print("D3 ejecución 1:")
printjson(db.clientes.updateOne(
 { _id: "C03" },
 { $addToSet: { preferencias: "japonesa" } }
))
print("D3 ejecución 2:")
printjson(db.clientes.updateOne(
 { _id: "C03" },
 { $addToSet: { preferencias: "japonesa" } }
))
printjson(db.clientes.findOne({ _id: "C03" }, { nombre: 1, preferencias: 1 }))
```

Respuesta :

```
{
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}

[
 {
  _id: 'P17',
  nombre: 'Arepa con hogao',
  tipo: 'acompañamiento',
  precio: 8000
 },
 { _id: 'P16', nombre: 'Bandeja paisa', tipo: 'plato', precio: 35000 }
]

D2 ejecución 1:
{
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
D2 ejecución 2:
{
 acknowledged: true,
 insertedId: null,
 matchedCount: 0,
 modifiedCount: 0,
```

```
  upsertedCount: 0
}
D3 ejecución 1:
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
D3 ejecución 2:
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}

{
  _id: 'C03',
  nombre: 'Valentina Ríos',
  preferencias: [
    'hamburguesas',
    'japonesa'
  ]
}

// Explicaciones:
//D1: el filtro exige el tipo: «plato». El R07 tiene dos productos, pero el P17 es de tipo
//«acompañamiento», por lo que solo hay un documento que cumple el filtro y solo se modifica
//uno.
//D2: matchedCount es el número de documentos que cumplen el filtro. En la segunda
//ejecución, PD018 ya está en_camino, por lo que el filtro estado: «pendiente» ya no encuentra
//nada y no se modifica nada. La condición del filtro hace que la operación sea segura para
//volver a ejecutarla.
//D3: en la segunda ejecución, se encuentra el documento (matchedCount: 1), pero $addToSet
//detecta que «japonesa» ya existe y no cambia nada (modifiedCount: 0).
```

Codigo :

```
// ===== PARTE D · Bloque 2 (D4 y D5) =====
use("saborya")

// D4. Registrar a Natalia Suárez sin duplicarla (upsert buscando por email; ejecutar dos veces)
print("D4 ejecución 1:")
printjson(db.clientes.updateOne(
  { email: "natalia.suarez@example.com" },
  { $setOnInsert: {
      _id: "C09",
      nombre: "Natalia Suárez",
```

```
    ciudad: "Medellín",
    puntos: 0,
    direcciones: []
  } },
  { upsert: true }
))
print("D4 ejecución 2:")
printjson(db.clientes.updateOne(
  { email: "natalia.suarez@example.com" },
  { $setOnInsert: {
    _id: "C09",
    nombre: "Natalia Suárez",
    ciudad: "Medellín",
    puntos: 0,
    direcciones: []
  } },
  { upsert: true }
))
print("D4 documentos con ese email: " + db.clientes.countDocuments({ email:
"natalia.suarez@example.com" }))
printjson(db.clientes.findOne({ email: "natalia.suarez@example.com" }))

// D5. Eliminar el campo volumen_ml del Té verde (P06)
printjson(db.productos.updateOne(
  { _id: "P06" },
  { $unset: { volumen_ml: "" } }
))
printjson(db.productos.findOne({ _id: "P06" }))
```

Respuesta :

```
D4 ejecución 1:
{
  acknowledged: true,
  insertedId: 'C09',
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
D4 ejecución 2:
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
D4 documentos con ese email: 1
{
  _id: 'C09',
  email: 'natalia.suarez@example.com',
```

```
  ciudad: 'Medellín',
  direcciones: [],
  nombre: 'Natalia Suárez',
  puntos: 0
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

{
  _id: 'P06',
  restaurante_id: 'R02',
  nombre: 'Té verde',
  tipo: 'bebida',
  precio: 7000,
  stock: 50,
  disponible: true,
  descripcion: 'Té verde japonés caliente'
}
```

Codigo :

```
// ===== PARTE D · Bloque 3 (D6 a D8) =====
use("saborya")

// D6. Eliminar reseñas con calificación menor a 3 (contar antes de borrar)
print("D6 reseñas a borrar: " + db.resenas.countDocuments({ calificacion: { $lt: 3 } }))
printjson(db.resenas.deleteMany({ calificacion: { $lt: 3 } }))

// D7. Eliminar pedidos cancelados de Medellín (contar antes de borrar)
print("D7 pedidos a borrar: " + db.pedidos.countDocuments({ estado: "cancelado",
"restaurante.ciudad": "Medellín" }))
printjson(db.pedidos.deleteMany({ estado: "cancelado", "restaurante.ciudad": "Medellín" }))

// D8. Verificación final
print("D8 restaurantes: " + db.restaurantes.countDocuments())
print("D8 clientes: " + db.clientes.countDocuments())
print("D8 productos: " + db.productos.countDocuments())
print("D8 pedidos: " + db.pedidos.countDocuments())
print("D8 resenas: " + db.resenas.countDocuments())
```

Respuesta :

```
{
  acknowledged: true,
  deletedCount: 1
}
```

POUPON Antoine

D7 pedidos a borrar: 1
{
  acknowledged: true,
  deletedCount: 1
}

